

Walchand College of Engineering, Sangli

Department of Computer Science and Engineering

Class: Final Year (Computer Science and Engineering)

Year: 2025-26

Semester: 1

Course: High Performance Computing Lab

Practical No. 4

Exam Seat No: 22510019

Batch : B6

Title of practical:

Study and Implementation of Synchronization

Problem Statement 1:

Analyze and implement a Parallel code for below programs using OpenMP considering synchronization requirements. (Demonstrate the use of different clauses and constructs wherever applicable)

Fibonacci Computation:

Screenshots:

```
PS C:\Users\asus> wsl
soham@Vivobook16x-Soham:/mnt/c/Users/asus$ cd /mnt/d/SEM\ VII/HPCL/Assignment4/
soham@Vivobook16x-Soham:/mnt/d/SEM VII/HPCL/Assignment4$ g++ -fopenmp ps1.cpp -o ps1
soham@Vivobook16x-Soham:/mnt/d/SEM VII/HPCL/Assignment4$ ./ps1
Enter no. of elements: 15
Computing series using 16 threads
series: 0 1 1 1 2 1 0 1 0 0 0 0 0 0 0 0
Time taken: 0.0054821 sec
soham@Vivobook16x-Soham:/mnt/d/SEM VII/HPCL/Assignment4$
```

Code :

```

long long fib(int n) {
    if (n <= 1) return n;

    long long x, y;

    // create parallel tasks only for bigger n to reduce overhead
    #pragma omp task shared(x) if(n > 20)
    x = fib(n - 1);

    #pragma omp task shared(y) if(n > 20)
    y = fib(n - 2);

    #pragma omp taskwait
    return x + y;
}

int main() {
    int n;
    cout << "Enter number of elements: ";
    cin >> n;

    double start = omp_get_wtime();

    long long result;
    #pragma omp parallel
    {
        #pragma omp single
        {
            cout << "Computing Fibonacci using "
                << omp_get_num_threads() << " threads\n";
            result = fib(n);
        }
    }

    double end = omp_get_wtime();

    cout << "Fibonacci(" << n << ") = " << result << "\n";
    cout << "Time taken: " << (end - start) << " sec\n";
    return 0;
}

```

Information:

$$F(n)=F(n-1)+F(n-2), F(0)=0, F(1)=1$$

This problem is sequential by nature because each value depends on previous ones. However, using OpenMP constructs like tasks and synchronization, we can parallelize the recursive computation and demonstrate how multiple threads work together safely.

Different constructs used in the program are :

1. **#pragma omp parallel** → creates a team of threads.
2. **#pragma omp single** → ensures only one thread starts the top-level recursion.
3. **#pragma omp task** → allows splitting work into subtasks.
4. **#pragma omp taskwait** → forces synchronization between subtasks.

Observations

1. Without synchronization (taskwait).. results would be incorrect due to incomplete calculations
2. So the recursive Fibonacci becomes much faster and achieves true parallel execution.

Problem Statement 2:

Analyze and implement a Parallel code for below programs using OpenMP considering synchronization requirements. (Demonstrate the use of different clauses and constructs wherever applicable)

Producer Consumer Problem

Screenshots:

```
PS C:\Users\asus> wsl
soham@Vivobook16x-Soham:/mnt/c/Users/asus$ cd /mnt/d/SEM\ VII/HPCL/Assignment4/
soham@Vivobook16x-Soham:/mnt/d/SEM VII/HPCL/Assignment4$ g++ -fopenmp ps2.cpp -o ps2
soham@Vivobook16x-Soham:/mnt/d/SEM VII/HPCL/Assignment4$ ./ps2
Producer produced: 1
Producer produced: 2
Producer produced: 3
Producer produced: 4
Producer produced: 5
Consumer consumed: 1
Consumer consumed: 2
Consumer consumed: 3
Consumer consumed: 4
Consumer consumed: 5
Producer produced: 6
Producer produced: 7
Producer produced: 8
Producer produced: 9
Producer produced: 10
Consumer consumed: 6
Consumer consumed: 7
Consumer consumed: 8
Consumer consumed: 9
Consumer consumed: 10
Total time: 0.000555023 sec
```

Code :

```

const int buffer = 5;
const int cnt = 10;

queue<int> buff;
int producer_item = 0;
int consumer_item = 0;

int main() {
    omp_set_num_threads(2);
    double start = omp_get_wtime();

    #pragma omp parallel sections shared(buff, producer_item, consumer_item)
    {
        #pragma omp section
        {
            while (producer_item < cnt) {
                #pragma omp critical
                {
                    if (buff.size() < buffer) {
                        int item = producer_item + 1;
                        buff.push(item);
                        producer_item++;
                        cout << "Producer produced: " << item << "\n";
                    }
                }
            }
        }
    }
}

```

```

        #pragma omp section
        {
            while (consumer_item < cnt) {
                #pragma omp critical
                {
                    if (!buff.empty()) {
                        int item = buff.front();
                        buff.pop();
                        consumer_item++;
                        cout << "Consumer consumed: " << item << endl;
                    }
                }
            }
        }
    }

    double end = omp_get_wtime();
    cout << "Total time: " << (end - start) << " sec\n";

    return 0;
}

```

Information:

Synchronization is required to ensure that..

The producer does not add items when the buffer is full.

& The consumer does not remove items when the buffer is empty.

Constructs Used :

#pragma omp parallel sections Divides work for the producer and consumer & Each section runs in a separate thread.

#pragma omp section : Assigns specific tasks to threads.

#pragma omp critical : Ensures mutual exclusion when accessing the shared buffer.

Analysis :

- Items are produced and consumed in order.
- Buffer size constraint (buff_SIZE = 5) is respected.
- Sometimes the producer runs ahead (produces multiple items before consumer catches up).
- Synchronization using critical ensures correctness (no data loss or corruption).
- Execution time is very small (0.000555 sec) → shows efficiency for small-scale demo.

Github Link: [🌐 HPCL/Assignment4 at main · Somzee5/HPCL](#)

Final Year: High Performance Computing Lab 2025-26 Sem I